



南开大学
Nankai University

南 开 大 学

数 学 科 学 学 院

机器视觉技术实验报告

深度学习手势识别

谢 博

年级：2024 级

专业：数学与人工智能

指导教师：王翔宇、孙明竹

2026 年 6 月 7 日

摘要

本文围绕机器视觉中的手势姿态识别任务，基于 PyTorch 搭建了一个六分类手势图像识别系统。实验数据集包含 A、B、C、Five、Point 和 V 六类手势，共 5040 张 PNG 图像。模型采用 ResNet50 作为主干网络，加载 ImageNet 预训练权重并替换末端全连接分类层，以迁移学习方式完成手势分类训练。训练阶段使用随机裁剪、随机水平翻转、颜色扰动和亮度扰动进行数据增强，优化器采用 AdamW，损失函数采用交叉熵损失，并使用余弦退火学习率调度策略。实验按 8:2 比例进行分层划分，训练 7 轮后模型在验证集上达到 99.90% 的最佳准确率，其中 B、C、Five、Point 和 V 五类在最终轮次达到 100.00% 的验证准确率。实验结果表明，基于预训练 ResNet50 的迁移学习方法能够有效提取手势图像的判别特征，并在当前数据集划分上取得较好的分类效果。本次作业相关代码已上传至 github，仓库地址为 <https://github.com/Zemiss/HandSignDL-Coursework>

关键字：手势识别；机器视觉；ResNet50；迁移学习；PyTorch；图像分类

目录

一、 实验目的	1
二、 实验原理	1
(一) 图像分类与卷积神经网络	1
(二) ResNet50 与迁移学习	1
(三) 损失函数与优化方法	1
三、 实验步骤	2
(一) 实验环境	2
(二) 数据集组成	2
(三) 实验流程	2
四、 程序代码	3
(一) 训练阶段图像增强	3
(二) ResNet50 模型构建	4
(三) 测试时增强推理	4
五、 实验结果显示	4
(一) 训练结果	4
(二) 各类别识别效果	5
六、 实验分析总结	6

一、实验目的

本实验以手势姿态图像分类为任务，目标是完成一个从数据读取、图像预处理、模型构建、训练验证到模型推理的完整深度学习实验流程。具体而言，实验需要实现对 A、B、C、Five、Point 和 v 六类手势图像的自动识别，并通过验证集准确率、损失曲线和各类别准确率评价模型性能。

通过本实验,可以进一步理解卷积神经网络在图像分类任务中的基本作用,掌握使用 ResNet50 预训练模型进行迁移学习的方法，熟悉 PyTorch 中数据集封装、数据增强、模型训练、权重保存和推理测试的实现方式。同时，实验也要求对训练结果进行分析，说明模型收敛趋势、类别识别效果以及当前实验设置可能存在的局限性。

二、实验原理

(一) 图像分类与卷积神经网络

图像分类的核心任务是将输入图像映射到预定义类别。传统方法通常依赖人工设计的边缘、纹理或形状特征，而卷积神经网络可以通过多层卷积、非线性激活和下采样结构自动学习由低级到高级的视觉特征。浅层卷积通常关注边缘、角点和局部纹理，深层卷积则能够组合出更抽象的结构特征，从而为分类器提供更强的判别信息。

本实验中的手势图像存在手部形状、指尖方向、局部弯曲程度和拍摄位置等差异，因此需要模型在局部纹理和整体轮廓两方面同时具备较好的表达能力。卷积神经网络适合处理这类二维图像数据，其参数共享和局部感受野机制能够降低参数规模，并提高对局部平移变化的鲁棒性。

(二) ResNet50 与迁移学习

ResNet50 是一种典型的残差卷积神经网络。普通深层网络在层数增加时容易出现梯度消失、训练退化等问题，残差网络通过引入 shortcut connection，使网络学习残差映射而不是直接学习完整映射，从而改善深层网络的可训练性 [2]。ResNet50 由多个残差瓶颈模块组成，具备较强的特征提取能力。

迁移学习的基本思想是将大规模数据集上学到的通用视觉特征迁移到新的目标任务中。本实验默认加载 ImageNet 预训练权重 [1]，再将 ResNet50 原本面向 1000 类分类的全连接层替换为六分类输出层。这样可以减少从零训练所需的数据量和训练时间，并提高小规模手势数据集上的收敛速度与泛化能力。

(三) 损失函数与优化方法

本实验采用交叉熵损失函数衡量预测类别分布与真实标签之间的差异。对于单张样本，若真实类别为 y ，模型对类别 k 的预测概率为 p_k ，则交叉熵损失可表示为：

$$L = -\log p_y. \quad (1)$$

该损失函数适用于多分类任务，能够促使模型提高真实类别的预测概率。

优化器采用 AdamW。AdamW 在 Adam 自适应学习率的基础上将权重衰减与梯度更新解耦，有助于改善正则化效果 [3]。训练过程中还使用 CosineAnnealingLR 学习率调度器，使学习率按照余弦曲线逐步调整，从而在训练后期获得更稳定的收敛效果。

三、实验步骤

(一) 实验环境

项目使用 Python 3.11 和 PyTorch 实现, 主要依赖包括 `torch`、`torchvision`、`Pillow`、`numpy`、`matplotlib` 和 `PyYAML`。训练脚本默认要求 CUDA 设备, 以便使用 GPU 加速 ResNet50 的训练。项目的默认配置文件为 `configs/default.yaml`, 训练入口为 `train.py`, 测试入口为 `test.py`, 核心数据处理与模型构建逻辑位于 `src/core.py`。

(二) 数据集组成

实验数据位于 `data/Hand_Posture_Hard_Stu` 目录下, 按类别子目录存放。数据集共包含 5040 张 PNG 图像, 六类样本数量如表1所示。

类别	A	B	C	Five	Point	V	合计
图像数量	1327	508	604	707	1394	500	5040

表 1: 手势姿态数据集类别分布

训练时按照 8:2 的比例进行分层划分, 保证每个类别都同时出现在训练集和验证集中。根据当前数据规模, 训练集约包含 4035 张图像, 验证集约包含 1005 张图像。由于各类别样本数量并不完全均衡, 实验在结果部分同时报告整体准确率和各类别准确率, 以便观察不同类别上的识别表现。

(三) 实验流程

输入图像首先被转换为 RGB 格式, 并统一处理为 224×224 尺寸。训练阶段采用随机裁剪、随机水平翻转、颜色扰动和亮度扰动, 以增强模型对位置、方向和光照变化的适应能力。验证阶段则使用固定缩放, 避免随机增强影响评估稳定性。所有图像最终按照 ImageNet 均值和标准差进行归一化, 使输入分布与预训练模型训练时更接近。

模型使用 `torchvision.models.resnet50` 构建。当本地存在 `model/resnet50-11ad3fa6.pth` 时, 程序优先加载本地 ImageNet 预训练权重; 否则使用 `torchvision` 默认预训练权重。随后将模型末端的全连接层替换为输出维度为 6 的线性层, 对应六类手势。

训练脚本首先读取配置文件, 设置随机种子, 然后构造训练集和验证集的 `DataLoader`。每个 epoch 中, 模型在训练阶段开启梯度计算并更新参数, 在验证阶段关闭梯度计算并统计损失、整体准确率和各类别准确率。若当前验证准确率超过历史最佳值, 程序会将模型权重保存到 `model/best_model.pth`。

Algorithm 1 手势识别模型训练流程

Input: 手势图像数据集、类别列表、训练配置

Output: 最佳模型权重、训练历史与训练曲线

- 1: 读取配置文件并设置随机种子
- 2: 构建训练集增强变换和验证集固定变换
- 3: 按类别分层划分训练集和验证集
- 4: 加载 ResNet50 预训练权重并替换分类层
- 5: 初始化交叉熵损失、AdamW 优化器和余弦退火调度器

```

6: for epoch = 1 to 7 do
7:   在训练集上前向传播、反向传播并更新参数
8:   在验证集上计算损失、整体准确率和各类别准确率
9:   if 当前验证准确率高于历史最佳值 then
10:    保存模型到 model/best_model.pth
11:   end if
12:   记录 batch 级训练损失和 epoch 级验证指标
13: end for
14: 根据训练历史绘制损失与准确率曲线

```

实验采用的主要训练参数如表2所示。

参数	取值	说明
输入尺寸	224 × 224	ResNet50 输入图像尺寸
训练轮数	7	默认配置中的 epoch 数
批大小	64	每个 batch 的图像数量
初始学习率	0.001	AdamW 初始学习率
验证集比例	0.2	分层划分验证集
优化器	AdamW	权重衰减为 10^{-4}
学习率调度	CosineAnnealingLR	训练过程中进行余弦退火
预训练权重	ImageNet ResNet50	用于迁移学习初始化

表 2: 主要训练参数

测试脚本会加载训练得到的 checkpoint，并递归读取测试目录中的常见图片格式。为了提高单张图片预测的稳定性，推理阶段采用测试时增强：先将图像缩放到比输入尺寸略大的尺度，再取四角裁剪和中心裁剪，并加入水平翻转视角，最后对多个视角的 logits 求平均后输出预测类别。

四、 程序代码

本实验的主要程序由 src/core.py、train.py 和 test.py 组成。其中，src/core.py 负责数据集封装、图像预处理、模型构建和 checkpoint 读写；train.py 负责训练、验证、结果记录和曲线绘制；test.py 负责加载模型并对测试图片进行推理。下面列出本实验中最关键的代码片段。

(一) 训练阶段图像增强

训练阶段图像增强

```

1 class TrainTransform:
2     def __init__(self, image_size: int = 64) -> None:
3         self.image_size = image_size
4
5     def __call__(self, image: Image.Image) -> torch.Tensor:
6         image = ensure_rgb(image).resize((self.image_size + 8, self.
            image_size + 8))

```

```

7         left = random.randint(0, 8)
8         top = random.randint(0, 8)
9         image = image.crop((left, top, left + self.image_size, top + self.
            image_size))
10        if random.random() < 0.5:
11            image = image.transpose(Image.FLIP_LEFT_RIGHT)
12        image = ImageEnhance.Color(image).enhance(random.uniform(0.85, 1.15))
13        image = ImageEnhance.Brightness(image).enhance(random.uniform(0.85,
            1.15))
14        return image_to_tensor(image)

```

(二) ResNet50 模型构建

ResNet50 六分类模型构建

```

1 def build_model(num_classes: int = 6, pretrained: bool = True,
2                 pretrained_weights_path: str | Path | None = None) -> nn.
    Module:
3     local_weights_path = Path(pretrained_weights_path) if
        pretrained_weights_path else None
4     if pretrained and local_weights_path and local_weights_path.exists():
5         model = resnet50(weights=None)
6         state_dict = torch.load(local_weights_path, map_location="cpu")
7         model.load_state_dict(state_dict)
8     else:
9         weights = ResNet50_Weights.DEFAULT if pretrained else None
10        model = resnet50(weights=weights)
11        model.fc = nn.Linear(model.fc.in_features, num_classes)
12    return model

```

(三) 测试时增强推理

测试时增强推理

```

1 def predict_image(model, image, image_size, device):
2     tensors = build_tta_tensors(image, image_size)
3     batch = torch.stack(tensors).to(device)
4     logits = model(batch)
5     return logits.mean(dim=0, keepdim=True)

```

五、 实验结果显示

(一) 训练结果

训练过程记录保存在 outputs/training_history.csv, 训练曲线保存在 outputs/training_curves.png。
本次实验共训练 7 轮, epoch 级结果如表3所示。

轮次	训练损失	训练准确率	验证损失	验证准确率
1	0.2593	91.25%	0.1300	97.01%
2	0.0607	98.36%	0.2138	92.34%
3	0.0602	97.99%	0.0700	98.11%
4	0.0510	98.59%	0.0150	99.40%
5	0.0096	99.68%	0.0019	99.90%
6	0.0034	99.93%	0.0031	99.90%
7	0.0018	99.93%	0.0021	99.90%

表 3: 训练与验证结果

从表3可以看出，模型在第 1 轮后已经达到 97.01% 的验证准确率，说明 ImageNet 预训练特征对手势图像分类任务具有较好的迁移效果。第 2 轮验证准确率下降到 92.34%，但随后验证损失和验证准确率迅速改善，第 5 轮达到 99.90% 的最佳验证准确率，并在第 6、7 轮保持稳定。训练损失从 0.2593 降至 0.0018，表明模型已经充分拟合当前训练划分。

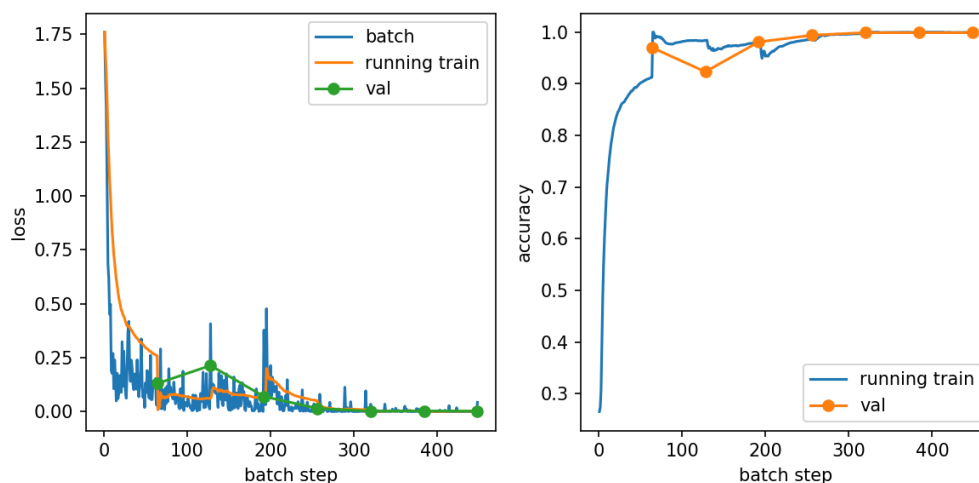


图 1: 训练损失、累计训练准确率与验证指标曲线

图1展示了 batch 级训练损失、累计训练损失和验证损失的变化趋势。整体来看，训练损失快速下降，后期趋于平稳；验证指标在第 3 轮之后明显改善，说明模型经过若干轮微调后能够更好地适应手势数据集。

(二) 各类别识别效果

第 7 轮各类别验证准确率如表4所示。

类别	验证准确率
A	99.62%
B	100.00%
C	100.00%
Five	100.00%
Point	100.00%
V	100.00%

表 4: 第 7 轮各类别验证准确率

结果显示, 除 A 类为 99.62% 外, 其余五类均达到 100.00% 的验证准确率。

六、 实验分析总结

本实验在当前验证集划分上取得了较高的准确率, 主要原因包括三个方面。第一, ResNet50 预训练权重提供了较强的通用视觉特征, 使模型不需要从零开始学习边缘和纹理等底层特征。第二, 训练数据增强增加了输入变化, 有助于提升模型对裁剪位置、左右方向和光照变化的鲁棒性。第三, AdamW 与余弦退火调度器使训练过程较为稳定, 后期验证损失保持在较低水平。

结合训练过程可知, 早期部分类别存在识别不稳定现象, 例如第 2 轮 A 类和 V 类准确率下降较明显, 但随着训练继续进行, 各类别性能均得到提升。这说明数据增强、预训练模型和优化策略共同提升了模型对手势轮廓和局部形态差异的识别能力。

同时也需要注意, 验证准确率接近 100% 并不意味着模型在所有真实场景中都能保持同等效果。当前训练集和验证集来自同一数据目录, 图像采集条件、背景和手势风格可能较为接近。如果要进一步验证实际应用能力, 应增加独立外部测试集, 覆盖更多拍摄角度、不同光照、不同背景和不同被试者。同时, 还可以加入混淆矩阵分析, 重点观察形状相近手势之间的错误类型。

本实验完成了基于 PyTorch 的手势姿态识别系统, 实现了数据集读取、图像预处理、ResNet50 迁移学习训练、验证评估、模型保存和测试推理等步骤。实验表明, 使用 ImageNet 预训练 ResNet50 作为主干网络, 可以在六类手势图像分类任务上快速收敛, 并在当前验证集划分上达到 99.90% 的最佳准确率。

从实验过程看, 迁移学习能够显著降低模型训练难度, 数据增强能够提升模型对输入变化的适应能力, 而分层划分和各类别准确率统计有助于更全面地评估模型表现。后续改进可以从三个方向展开: 一是构建独立测试集, 检验模型在真实场景中的泛化能力; 二是引入混淆矩阵、召回率和 F1 值等更多评价指标; 三是比较 ResNet18、MobileNet、EfficientNet 等不同主干网络, 在准确率、推理速度和模型大小之间寻找更适合部署的方案。

参考文献

- [1] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255. IEEE, 2009.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [3] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

NIJUB